

Recursion

example of reduction

how do I know that an algorithm is recursive?

calls itself

→ reduce a problem to itself

ex sorting an array

A = ALG OR IT HMS
L R

$S_L = \text{ALGOR}$

$S_R = \text{HIMST}$

$\text{merge}(S_L, S_R) = \text{AGHILMORST}$

$\text{mergesort}(A):$

if $\text{len}(A) > 1:$

→ $S_L = \text{mergesort}(L)$

→ $S_R = \text{mergesort}(R)$

→ $\text{return merge}(S_L, S_R)$

else:

$\text{return } A$

$\text{merge}(\text{sorted arrays } B, C):$

$S = []$

while B, C not empty:

add the smaller of B or C's first elt to S

remove said elt from B or C

ex

$B = S_L = \cancel{A} \cancel{G} \cancel{K} \cancel{O}$

$C = S_R = \cancel{A} \cancel{X} \cancel{M} \cancel{S}$

$S = A G H I L O R S T$

With table

merge

Describe a best-case input to ~~IsPrime~~.

merge

Give a function $f(n)$ so that the best-case runtime of ~~IsPrime~~ is $\Theta(f(n))$.

Describe a worst-case input to ~~IsPrime~~.

merge

Give a function $f(n)$ so that the worst-case runtime of ~~IsPrime~~ is $\Theta(f(n))$.

merge

worst = best

runtime of merge is $\Theta(n)$

if time :

mergesort

Describe a best-case input to ~~IsPrime~~.

mergesort

Give a function $f(n)$ so that the best-case runtime of ~~IsPrime~~ is $\Theta(f(n))$.

Describe a worst-case input to ~~IsPrime~~.

mergesort

mergesort

Give a function $f(n)$ so that the worst-case runtime of ~~IsPrime~~ is $\Theta(f(n))$.

Analyze recursive algs recursively!

let $T(n)$ = runtime of mergesort on input of size n

if $n=1$:

$$T(n) = \Theta(1)$$

if $n > 1$: $2 \cdot T(\frac{n}{2})$

$T(n) =$ time to make 2 recursive calls to mergesort on input of size $n/2$

+

time to merge two sorted arrays of size $n/2$

$$\begin{aligned} &\downarrow \\ &\Theta\left(\frac{n}{2}\right) \\ &= \Theta(n) \\ &\uparrow \end{aligned}$$

$$T(n) = \begin{cases} 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1 \\ \Theta(1) & \text{if } n = 1 \end{cases}$$

Want: closed form of $T(n)$

$\Rightarrow$ recursion trees

mergesort (A of length n):

if $n > 1$:
 $S_L = \text{mergesort}(\text{L half of } A)$
 $S_R = \text{mergesort}(\text{R half of } A)$
return merge(S_L, S_R)

else:
return A